Yau-Yau Filter: Algorithm Theory and Code

Yu Wang

Beijing Institute of Mathematical Sciences and Applications (BIMSA)

October 5, 2024

Historical Background

- The concept of filtering has its origins in signal processing and statistics.
  - 1940s: Wiener filter for continuous-time systems;
  - 1960s: Kalman filter for discrete-time systems;
  - Modern era: Particle filters, Yau-Yau filter and adaptive filtering techniques.
- Nonlinear Filtering Problems (NFP):
  - $\mathbf{x}(t)$: signal / state;
  - $\mathbf{y}(t)$: observation / measurement;
  - Goal: Estimate the state $\mathbf{x}(t)$ by a given history of observations:

$$\{\mathbf{y}(\tau)|\tau \in [0,t]\}, \quad t \in (0,T].$$

Signal–Observation Model

- Consider the classical NFP for a signal-observation model:

$$\begin{aligned} \mathrm{d}\mathbf{x}(t) &= \mathbf{f}(\mathbf{x}(t))\mathrm{d}t + \mathrm{d}\mathbf{v}(t), \quad \mathbf{x}(0) = \mathbf{x}_0. \\ \mathrm{d}\mathbf{y}(t) &= \mathbf{h}(\mathbf{x}(t))\mathrm{d}t + \mathrm{d}\mathbf{w}(t), \quad \mathbf{y}(0) = 0. \end{aligned}$$

where

- $\mathbf{x}(t) = (x_1(t), \ldots, x_N(t))^\top \in \mathbb{R}^N$: state.
- $\mathbf{y}(t) = (y_1(t), \ldots, y_M(t))^\top \in \mathbb{R}^M$: measurement.
- $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_N(\mathbf{x}))^\top \in \mathbb{R}^N$: (nonlinear) drift term.
- $\mathbf{h}(\mathbf{x}) = (h_1(\mathbf{x}), \ldots, h_M(\mathbf{x}))^\top \in \mathbb{R}^M$: (nonlinear) observation term.
- $\mathbf{v}(t) \in \mathbb{R}^N, \mathbf{w}(t) \in \mathbb{R}^M$: independent Brownian motions.

Duncan–Mortensen–Zakai (DMZ) equation

- Let $\rho(t, s)$ denote the conditional probability density of $\mathbf{x}(t)$ given $\mathbf{y}(\tau)$.
- $\rho(t, s)$ derived from normalizing $\sigma(t, s)$ that satisfies the **DMZ equation**:

$$\mathrm{d}\sigma(t,s) = L_0\sigma(t,s)\mathrm{d}t + \sum_{i=1}^{n} L_i\sigma(t,s)\mathrm{d}y_i(t), \quad \sigma(0,s) = \sigma_0,$$

$$L_0 = \frac{1}{2}\sum_{i=1}^{n}\frac{\partial^2}{\partial s_i^2} - \sum_{i=1}^{n} f_i\frac{\partial}{\partial s_i} - \sum_{i=1}^{n}\frac{\partial f_i}{\partial s_i} - \frac{1}{2}\sum_{i=1}^{m} h_i^2.$$

  - $\{L_i\}_{i=1}^m$ is the zero degree differential operator of multiplication by h_i.
- The central problem of nonlinear filtering theory is to solve the DMZ equation in real-time and in a memoryless way.


Duncan, Probability Density for Diffusion Processes with Applications to Nonlinear Filtering Theory, Ph.D. thesis, Stanford University, Stanford, CA, 1967.


Mortensen, Optional Control of Continuous Time Stochastic Systems, Ph.D. thesis, University of California, Berkeley, CA, 1966.


Zakai, On the optimal filtering of diffusion processes, *Z. Wahrsch. Verw. Gebiete*, 1969.

The Robust DMZ Equation

- Define the new unnormalized density as $u(t,s) = e^{\left(-\sum_{i=1}^m h_i(x) y_i(t)\right)} \sigma(t,s)$
- Let $u(0,s) = \sigma_0(s)$, Davis given the robust DMZ equation is given by

$$\frac{\partial u}{\partial t}(t,s) = L_0 u(t,s) + \sum_{i=1}^m y_i(t)[L_0, L_i]u(t,s) + \frac{1}{2}\sum_{i,j=1}^m y_i(t)y_j(t)[[L_0, L_i], L_j]u(t,s).$$

  - $[\cdot,\cdot]$ denotes the Lie bracket.

Davis, . , *Z. Wahrsch. Verw. Gebiete*, 1980.

- In the work of Yau and Yau, the robust DMZ equation is reformulated as

$$\begin{aligned}\frac{\partial u}{\partial t}(t,s) = &\frac{\Delta u(t,s)}{2} + (-f(s) + \nabla K(t,s)) \cdot \nabla u(t,s) \\ &+ \left(-\nabla \cdot f(s) - \frac{|h(s)|^2}{2} + \frac{1}{2}\Delta K(t,s)\right) u(t,s) \\ &- f(s) \cdot \nabla K(t,s) + \frac{1}{2}|\nabla K(t,s)|^2 u(t,s).\end{aligned}$$

  - $K = \sum_{i=1}^m y_i(t) h_i(s)$, $f = (f_1, \ldots, f_n)^\top$, and $h = (h_1, \ldots, h_m)^\top$.

Yau and Yau, . , *Mathematical Research Letters*, 2000.

The Kolmogorov Equation

- Yau and Yau showed that under some mild conditions, the DMZ equation admits a unique nonnegative solution $u(\tau, s)$ can be computed by $\tilde{u}_k(\tau_k, s)$.
- $\tilde{u}_k(t, s)|_{[\tau_k - 1, \tau_k]}$ satisfies the Kolmororov equation

$$\begin{aligned}
\frac{\partial \tilde{u}_k}{\partial t}(t, s) &= \frac{1}{2}\Delta \tilde{u}_k - \mathbf{f}(s) \cdot \nabla \tilde{u}_k - \left(\nabla \cdot \mathbf{f} + \frac{1}{2}|\mathbf{h}|^2 \right) \tilde{u}_k, \quad t \in [\tau_{k-1}, \tau_k], \\
\tilde{u}_k(\tau_{k-1}, s) &= \exp\left\{ (\mathbf{y}(\tau_{k-1}) - \mathbf{y}(\tau_{k-2})) \cdot \mathbf{h}(x) \right\} \tilde{u}_{k-1}(\tau_{k-1}, s), \\
\tilde{u}_1(0, s) &= \sigma_0(s) \exp\{ \mathbf{y}(0) \cdot \mathbf{h}(x) \}, \quad k = 2, \ldots, N_\tau.
\end{aligned}$$

- Then, $u(\tau_k, s) = \exp\left(-\sum_{j=1}^{m} y_j(\tau_{k-1}) h_j(x) \right) \tilde{u}_k(\tau_k, s)$.
- The estimate of the state is computed by $\mathbf{x}(t) \approx \int_\Omega s \mathrm{d}u(t, s)$

Algorithm Principle

1. Consider the classical NFP for a signal-observation model:

$$\begin{cases} \mathrm{d}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t)) + \mathrm{d}\mathbf{v}(t), \\ \mathrm{d}\mathbf{y}(t) = \mathbf{h}(\mathbf{x}(t)) + \mathrm{d}\mathbf{w}(t). \end{cases}$$

2. When $\mathbf{y}(\tau_k)$ is observed, update $u(\tau_k, s)$ at $t = \tau_k$:

$$\exp\left\{[\mathbf{y}(\tau_k) - \mathbf{y}(\tau_{k-1})] \cdot \mathbf{h}(s)\right\} u(\tau_k, s).$$

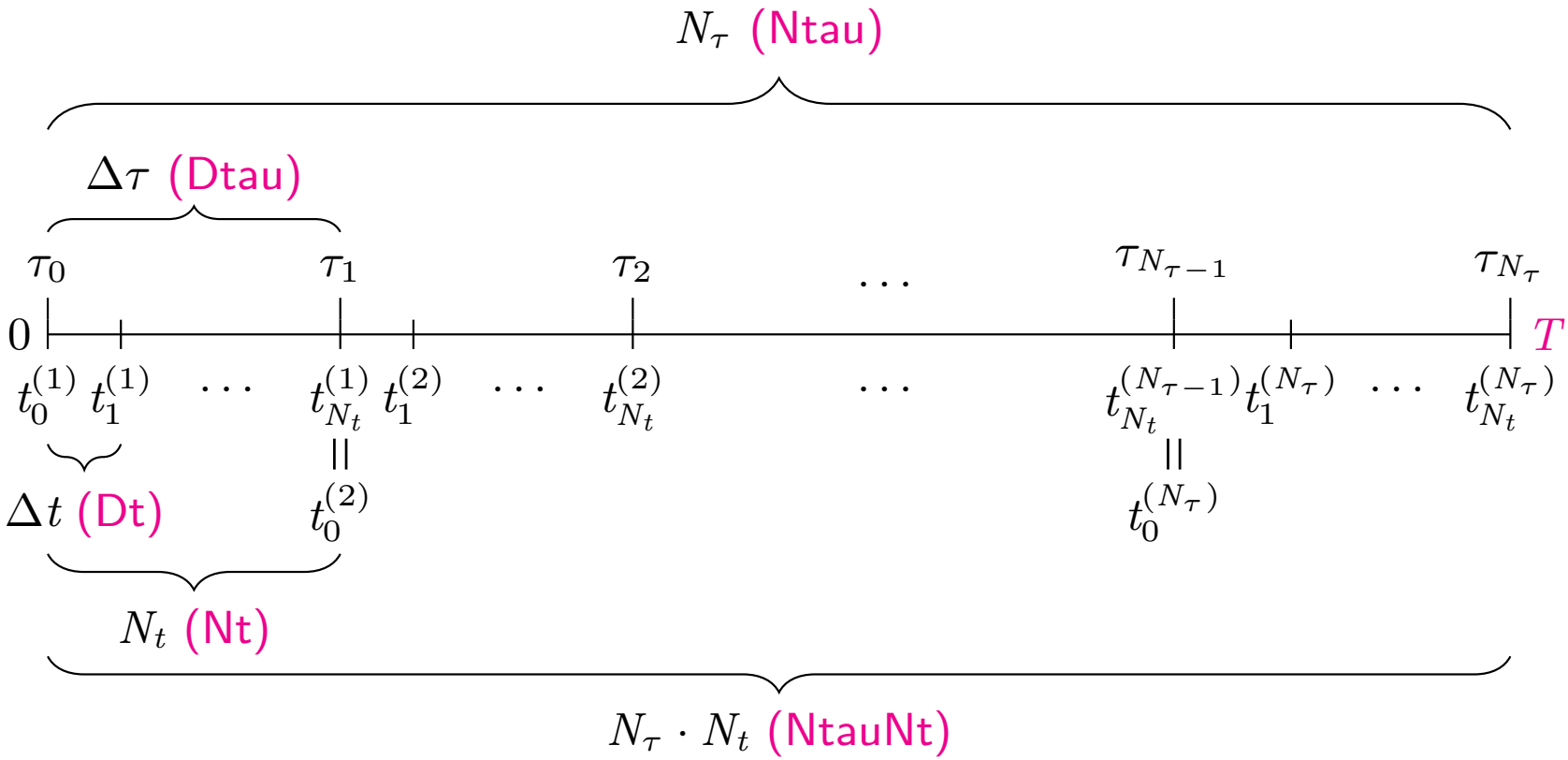
3. By Yau-Yau method, the NFP can be reduced into a Kolmogorov PDE:

$$\frac{\partial u}{\partial t}(t,s) = \frac{1}{2}\Delta u(t,s) - \mathbf{f}(s) \cdot \nabla u(t,s) - \left(\nabla \cdot \mathbf{f}(s) + \frac{1}{2}\|\mathbf{h}(s)\|_2^2\right) u(t,s)$$

   and the initial $u(0,s) = \sigma_0(s)$ is a given PDF.

4. The estimate of the state is computed by $x(t) \approx \int_\Omega s(t,s)$.

时间离散化




```
T       = 20;          % 总时间
Dt      = 0.001;       % 时间步长
Dtau    = 5*dt;        % 观测时间步长
Ntau    = T/dtau;      % 观测次数
Nt      = dtau/dt;     % 一次观测内状态演化次数
NtauNt  = T/dt;        % 总时间内状态演化次数
```

Generate States

1. Consider the classical NFP for a signal-observation model:

$$\begin{cases} \mathrm{d}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t)) + \mathrm{d}\mathbf{v}(t), \\ \mathrm{d}\mathbf{y}(t) = \mathbf{h}(\mathbf{x}(t)) + \mathrm{d}\mathbf{w}(t). \end{cases}$$

▷ We use the Euler forward method to generate states and observations:

$$\begin{cases} \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{f}(\mathbf{x}_k)\Delta\tau + \mathbf{v}\sqrt{\Delta\tau}, \\ \mathbf{y}_{k+1} = \mathbf{y}_k + \mathbf{h}(\mathbf{x}_k)\Delta\tau + \mathbf{w}\sqrt{\Delta\tau}, \end{cases}$$

where $\mathbf{x}_0$ is the initial vector, $\mathbf{y}_0 = 0$, $\Delta\tau$ is the time step.

```
for t = 1:NtauNt
  state(t+1,:) = state(t,:) + (f(state(t,:))*Dt) + (randn(1,Dim)*sqdT);
end
for t = 1:NtauNt
  obser(t+1,:) = obser(t,:) + (h(state(t,:))*Dt) + (randn(1,Dim)*sqdT);
end
y = obser(1:Nt:end,:);
```

Generate States

Ex. $T = 20, \quad \Delta t = 0.001, \quad \Delta\tau = 0.005.$
$N_t = 5, \quad N_\tau = 4000, \quad N_\tau \cdot N_t = 20000, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$

```
Dim    = 1;
T      = 20;
Dt     = 0.001;
Dtau   = 5*Dt;
Ntau   = T/Dtau;
Nt     = Dtau/Dt;
NtauNt = T/Dt;

f  = @(x)   cos(x);
df = @(x) - sin(x);
h  = @(x)   x.^3;
```

Generate States

Ex. $T = 20, \quad \Delta t = 0.001, \quad \Delta\tau = 0.005.$
$N_t = 5, \quad N_\tau = 4000, \quad N_\tau \cdot N_t = 20000, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$

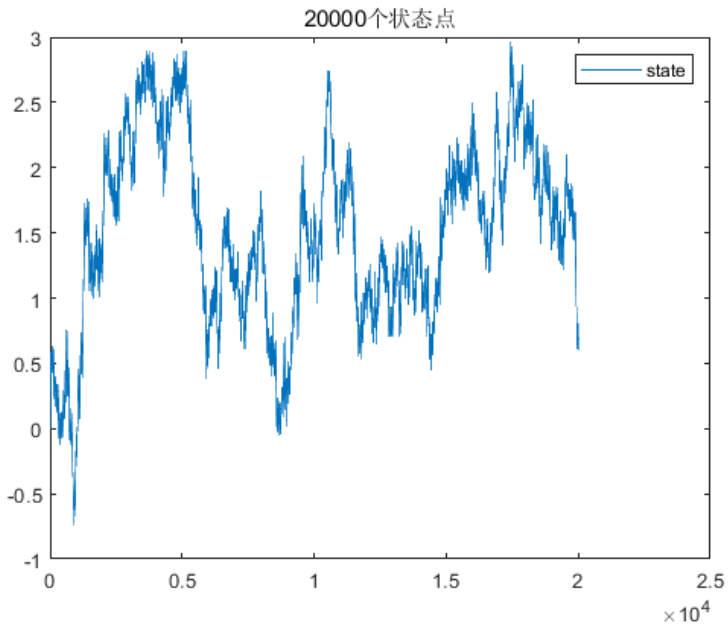
```
[state, obser, seed] = SimulateStateObser(Dt, NtauNt, f, h, Dim);
```

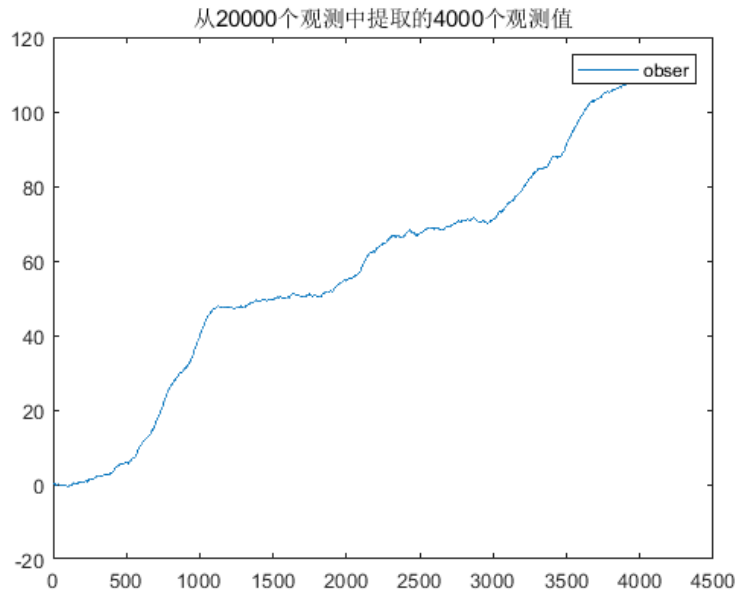
```
function [state, obser, s] = SimulateStateObser(Dt, NtauNt, f, h, Dim)
state = zeros(NtauNt, Dim);
obser = zeros(NtauNt, Dim);
sqDt = sqrt(Dt);
seed = rng('default');
for t = 1:NtauNt
  state(t+1,:) = state(t,:) + (f(state(t,:))*Dt) + (randn(1,Dim)*sqDt);
end
for t = 1:NtauNt
  obser(t+1,:) = obser(t,:) + (h(state(t,:))*Dt) + (randn(1,Dim)*sqDt);
end
```

```
y = obser(1:nT:end,:);
```

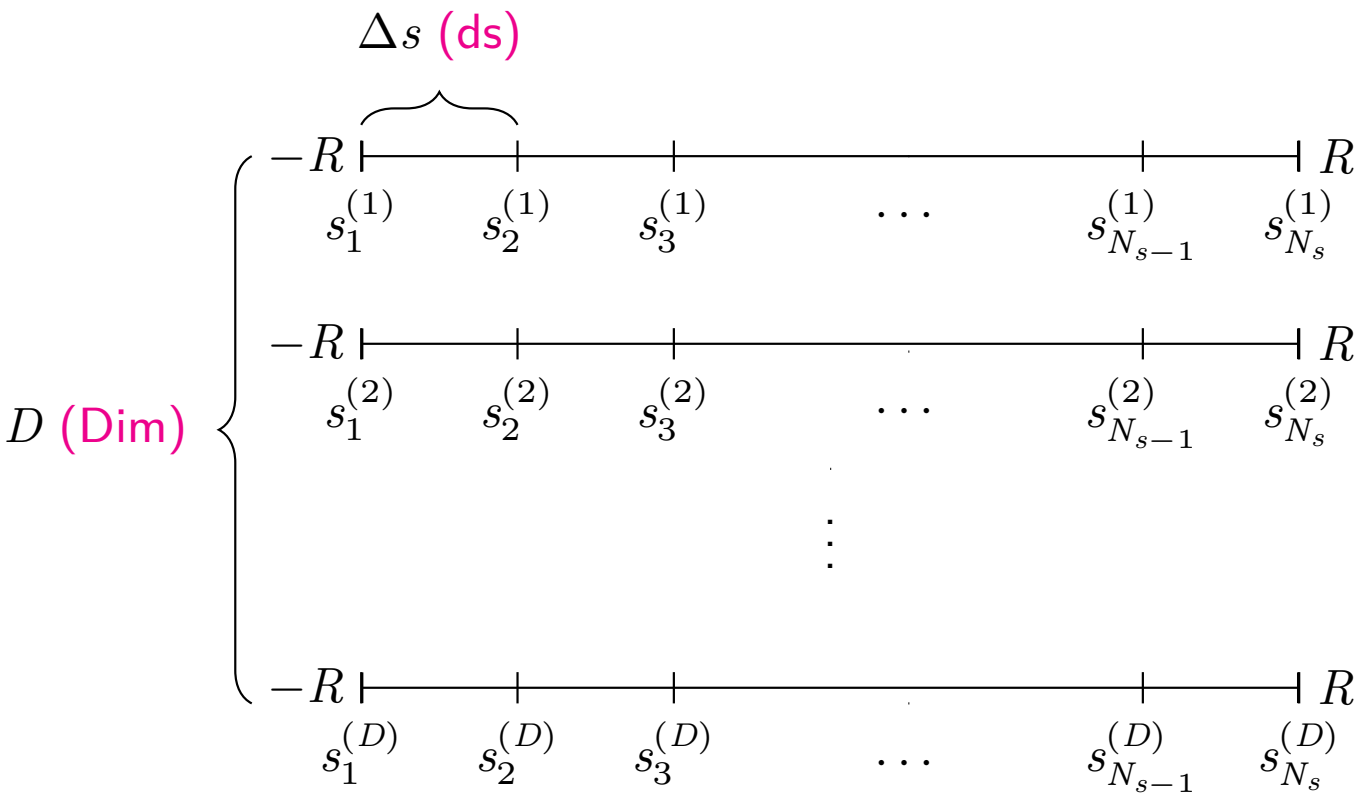
生成状态

Ex. $T = 20, \quad \Delta t = 0.001, \quad \Delta\tau = 0.005.$
$N_t = 5, \quad N_\tau = 4000, \quad N_\tau \cdot N_t = 20000, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$





Uniform Partition of $[-R, R]^{(D)}$




```
Dim     = 1;
ds      = 0.5;    % Size of Space Steps
rs = [min(min(state)), max(max(state))];
s = (rs(1):ds:rs(2)+ds).';
```

Finite Difference Schemes

Let $\{t_n\}_{n=1}^{N_t}$ and $\{s_j\}_{j=1}^{N_s}$ be a uniform partition of $[0, T]$ and $[-R, R]$ with step size Δt and Δs, respectively. Then

$$\begin{aligned}
\frac{\partial u}{\partial s^2}(t_n, s_j) &= \frac{\alpha}{(\Delta s)^2}\left(u(t_{n+1}, s_{j+1}) - 2u(t_{n+1}, s_j) + u(t_{n+1}, s_{j-1})\right) \\
&\quad + \frac{1-\alpha}{(\Delta s)^2}\left(u(t_n, s_{j+1}) - 2u(t_n, s_j) + u(t_n, s_{j-1})\right) + \mathcal{O}\left((\Delta s)^2\right), \\
\frac{\partial u}{\partial s}(t_n, s_j) &= \frac{\beta}{2\Delta s}\left(u(t_{n+1}, s_{j+1}) - u(t_{n+1}, s_{j-1})\right) \\
&\quad + \frac{1-\beta}{2\Delta s}\left(u(t_n, s_{j+1}) - u(t_n, s_{j-1})\right) + \mathcal{O}(\Delta s).
\end{aligned}$$

1. Explicit Euler Method: $\alpha = 0$, $\beta = 0$.
2. Implicit Euler Method: $\alpha = 1$, $\beta = 1$.
3. Quasi-Implicit Euler Method (QIEM): $\alpha = 1$, $\beta = 0$.
4. Crank-Nicolson Method: $\alpha = \frac{1}{2}$, $\beta = \frac{1}{2}$.

QIEM for Kolmogorov Equation

$$\frac{\partial u}{\partial t}(t,s) = \frac{1}{2}\Delta u(t,s) - \mathbf{f}(s)\cdot\nabla u(t,s) - \left(\nabla\cdot\mathbf{f}(s) + \frac{1}{2}\|\mathbf{h}(s)\|_2^2\right) u(t,s)$$

▷ By using the QIEM, we have

$$\frac{\partial^2 u}{\partial s^2}(t_n^{(k)}, s_j) \approx \frac{u(t_{n+1}^{(k)}, s_{j+1}) - 2u(t_{n+1}^{(k)}, s_j) + u(t_{n+1}^{(k)}, s_{j-1})}{(\Delta s)^2},$$

$$\frac{\partial u}{\partial s}(t_n^{(k)}, s_j) \approx \frac{u(t_n^{(k)}, s_{j+1}) - u(t_n^{(k)}, s_{j-1})}{2(\Delta s)},$$

$$\frac{\partial u}{\partial t}(t_n^{(k)}, s) \approx \frac{u(t_{n+1}^{(k)}, s) - u(t_n^{(k)}, s)}{\Delta t}, \quad \text{for } t_n \in [\tau_{k-1}, \tau_k].$$

Yueh, Lin and Yau, An efficient algorithm of YauYau method for solving nonlinear filtering problems, *Communications in Information and Systems*, 2014.

QIEM for Kolmogorov Equation

$$\frac{\partial u}{\partial t}(t,s) = \frac{1}{2}\Delta u(t,s) - \mathbf{f}(s)\cdot\nabla u(t,s) - \left(\nabla\cdot\mathbf{f}(s) + \frac{1}{2}\|\mathbf{h}(s)\|_2^2\right) u(t,s)$$

▷ The QIEM for Kolmogorov equation is formulated as

$$\frac{u(t_{n+1}^{(k)}, s_j) - u(t_n^{(k)}, s_j)}{\Delta t} = \frac{1}{2}\mathbf{L}_{N_s}^{(D)} u(t_{n+1}^{(k)}, s_j) + \left(\mathbf{K}_{N_s}^{(D)} + \mathbf{Q}_{N_s}^{(D)}\right) u(t_n^{(k)}, s_j),$$

where $\mathbf{Q}_{N_s}^{(D)} = \operatorname{diag}\left\{-\nabla\cdot\mathbf{f}(s_j) + \frac{1}{2}\|\mathbf{h}(s_j)\|_2^2\right\}_{j=1}^{N_s^{(D)}}$,

$$\mathbf{L}_{N_s} = \frac{1}{(\Delta s)^2}\begin{bmatrix} -2 & 1 & & \\ 1 & -2 & \ddots & \\ & \ddots & \ddots & 1 \\ & & 1 & -2 \end{bmatrix} \quad \text{and} \quad \mathbf{K}_{N_s} = \frac{1}{2(\Delta s)}\begin{bmatrix} 0 & 1 & & \\ -1 & 0 & \ddots & \\ & \ddots & \ddots & 1 \\ & & -1 & 0 \end{bmatrix}$$

▷ It can be solved by the linear system

$$\left[I_{N_s}^{(D)} - \frac{\Delta t}{2}\mathbf{L}_{N_s}^{(D)}\right] u^{n+1} = \left[I_{N_s}^{(D)} + \Delta t\left(\mathbf{K}_{N_s}^{(D)} + \mathbf{Q}_{N_s}^{(D)}\right)\right] u^n$$

DST for Solving the Linear System

$$\left[I_{N_s}^{(D)} - \frac{\Delta t}{2}\mathbf{L}_{N_s}^{(D)}\right] u^{n+1} = \left[I_{N_s}^{(D)} + \Delta t\left(\mathbf{K}_{N_s}^{(D)} + \mathbf{Q}_{N_s}^{(D)}\right)\right] u^n = b$$

The spectral decomposition of the matrix $\left[I_{(N_s)}^{N} - \frac{\Delta t}{2} L_{N_s}^{(N)}\right]$ in the linear system is

$$\left[I_{(N_s)}^{N} - \frac{\Delta t}{2} L_{N_s}^{(N)}\right] = W_{N_s}^{(N)} \Lambda_{N_s} \left(W_{N_s}^{(N)}\right)^*$$

Then, we can express the solution of the linear system $\left[I_{N_s}^{(D)} - \frac{\Delta t}{2}\mathbf{L}_{N_s}^{(D)}\right] u = b$ as

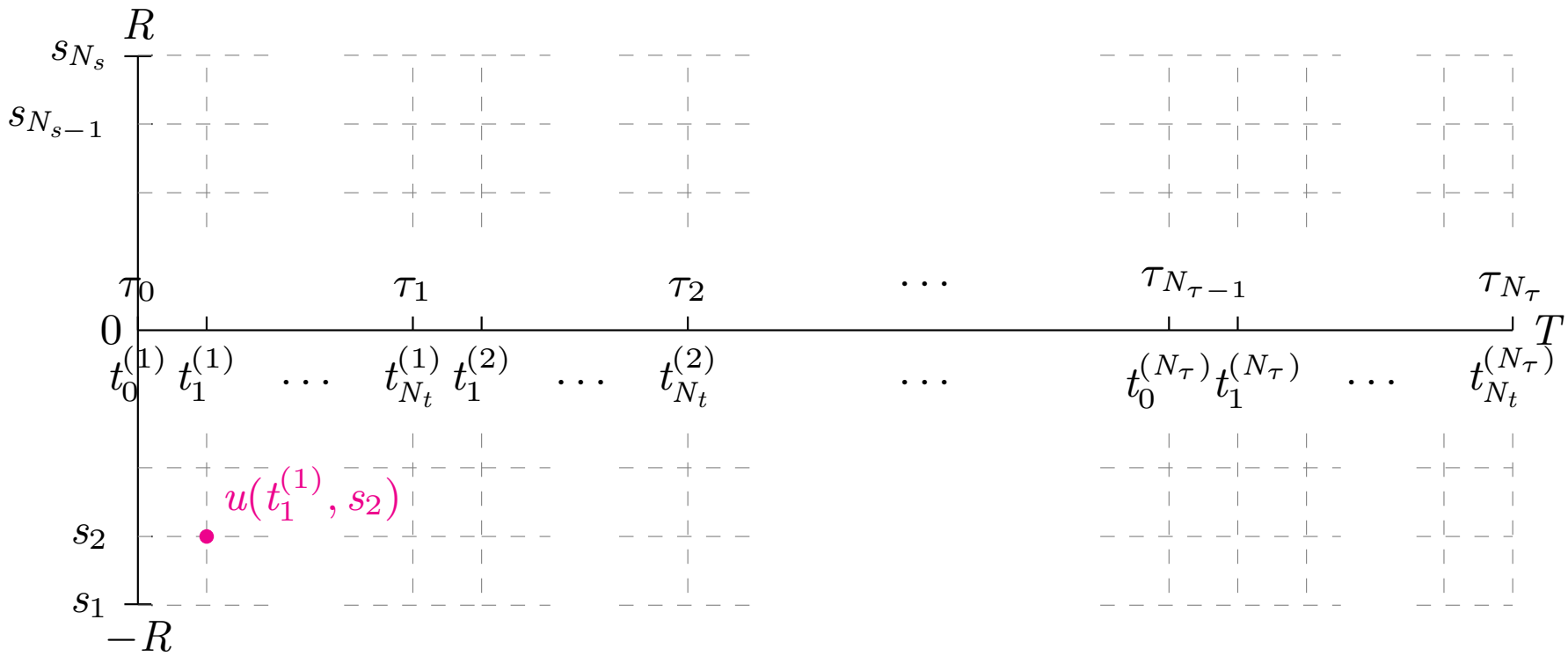
$$u^{n+1} = W_{N_s}^{(D)} \left(\Lambda_{N_s}^{-1} \left(\left(W_{N_s}^{(D)}\right)^* b\right)\right)$$

where

$$W_{N_s}^{(D)} = \otimes_{d=1}^{D} W_{N_s}, \quad \Lambda_{N_s}^{D}$$

and $W_{N_s} = \left[\sqrt{\frac{2}{N_s+1}} \sin\left(\frac{ij\pi}{2(N_s+1)}\right)\right]_{i,j=1}^{N_s}, \Lambda_{N_s}^{-1} = \operatorname{diag}\left(1 - \frac{2\Delta t}{(\Delta s)^2} \sin^2\left(\frac{i\pi}{2(N_s+1)}\right)\right)_{i=1}^{N_s}$.

时空离散化




```
rx = [min(min(state)), max(max(state))];
x = (rx(1):ds:rx(2)+ds).';
```

Construct Matrix

The Kolmogorov PDE can be written in the form

$$\frac{\partial u}{\partial t} = \frac{1}{2}\Delta u + p(x) \cdot \nabla u + q(x) \cdot u.$$

Based on the quasi-implicit Euler method **(QIEM)**, it can be discretized as

$$\frac{du_i(t)}{dt} = \frac{1}{2}\left(\frac{u_{i+1}(t) - 2u_i(t) + u_{i-1}(t)}{\Delta x^2}\right) + p(x_i)\left(\frac{u_{i+1}(t) - u_{i-1}(t)}{2\Delta x}\right) + q(x_i)u_i(t),$$

or briefly,

$$\frac{dU(t)}{dt} = A\,U(t).$$

From the Euler forward method, we have

$$U^{n+1} = U^n + \Delta t \cdot A\,U^{n+1}$$

or equivalently,

$$U^{n+1} = B \cdot U^n$$

where

$$B = (I - \Delta t \cdot A)^{-1}$$

can be simplified using the Discrete Sine Transform **(DST)**.

KolmogorovEW & LaplacianEW

Ex. $[0, T] = [0, 20], \quad \Delta\tau = 5\Delta t, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$

```
x = (rX(1):dX:rX(2)+dX).'; % 离散化后的空间网格点
[Lambda, B, x, n] = KolmogorovEW(Dim, dT, x, f, df, h);
```

```
function [Lambda, B, x, n] = KolmogorovEW(Dim, dT, x, f, df, h)
dX = x(2)-x(1); % 空间步长
n = size(x, 1); % 矩阵维度
p = @(x) -f(x);
q = @(x) -( sum(df(x), 2) + 0.5 * ( sum(h(x).^2, 2) ) );
e = ones(n,1);  % 构建稀疏矩阵
d = 0.5/dX * spdiags([-e, e], [-1, 1], n, n);   % 中央差分法离散化

P = spdiags(p(x), 0, n, n); % 对流项的对角矩阵
Q = spdiags(q(x), 0, n, n); % 零阶项的对角矩阵
Lambda = 1/(dX*dX) * LaplacianEW(Dim, n); % 离散化的拉普拉斯算子矩阵
% d = -4*sin((1:n).'*pi/(2*n+2)).^2;
Lambda = 1-0.5*dT*Lambda;    % 修正矩阵,增加稳定性
FW = P*d + Q;                % 对流项和零阶项的组合矩阵
B = speye(n) + dT*FW;        % 更新矩阵, 包含漂移项和零阶项信息
```

Solve Kolmogorov Equations

```
sigma0 = exp( -10 * ( sum(x.^2, 2) ) );  % 初始化高斯分布
Iu     = zeros((nTau-1)*nT+1, Dim);      % 存储每个时间步的状态估计的矩阵
Idx    = 1;                              % 跟踪当前计算的时间步
U      = sigma0;                         % 状态的初始估计
U      = U / sum(U);                     % 对 U 进行归一化
Iu(Idx,:) = sum(U(:,ones(Dim,1)).*x, 1); % 初始状态估计，对 U 加权平均后的期望

% 结合观测数据和状态模型，逐步更新，最终，Iu 存储所有时间点上的状态估计结果
for jj = 1:nTau % 大时间步迭代
    Idx = Idx + 1;
    if jj == 1
        tmp = y(jj,:);                % 当前的观测数据
    else
        tmp = y(jj,:) - y(jj-1,:);    % 增量更新状态
    end
    % 更新概率密度函数，结合了新的观测信息
    U = NormalizedExp( sum( h(x).*tmp(ones(size(x,1),1), :) ,2) ) .* U;
    U = DST_Solver(Dim, Lambda, B, U, n);    % 求解 DST，更新概率密度函数 U
    U = U / sum(U);                          % 再次归一化
    Iu(Idx,:) = sum(U(:,ones(Dim,1)).*x, 1); % 更新状态估计
    toc
    for ii = 2:nT   % 小时间步迭代
        Idx = Idx + 1;
        U = DST_Solver(Dim, Lambda, B, U, n);
        Iu(Idx,:) = sum(U(:,ones(Dim,1)).*x, 1);
        toc
    end % 通过多个小时间步逐步推进状态的演化
end % 大时间步上，结合新的观测数据进行校正
```

update solution

3. When $\mathbf{y}(\tau_k)$ is observed, update $u(\tau_k, s)$ at $t = \tau_k$:

$$\exp\left\{[\mathbf{y}(\tau_k) - \mathbf{y}(\tau_{k-1})] \cdot \mathbf{h}(s)\right\} u(\tau_k, s).$$

```
tmp = y(jj,:) - y(jj-1,:);
U = NormalizedExp( sum( h(x).*tmp(ones(size(x,1),1), :) ,2) ) .* U;
U = DST_Solver(Dim, Lambda, B, U, n);
```

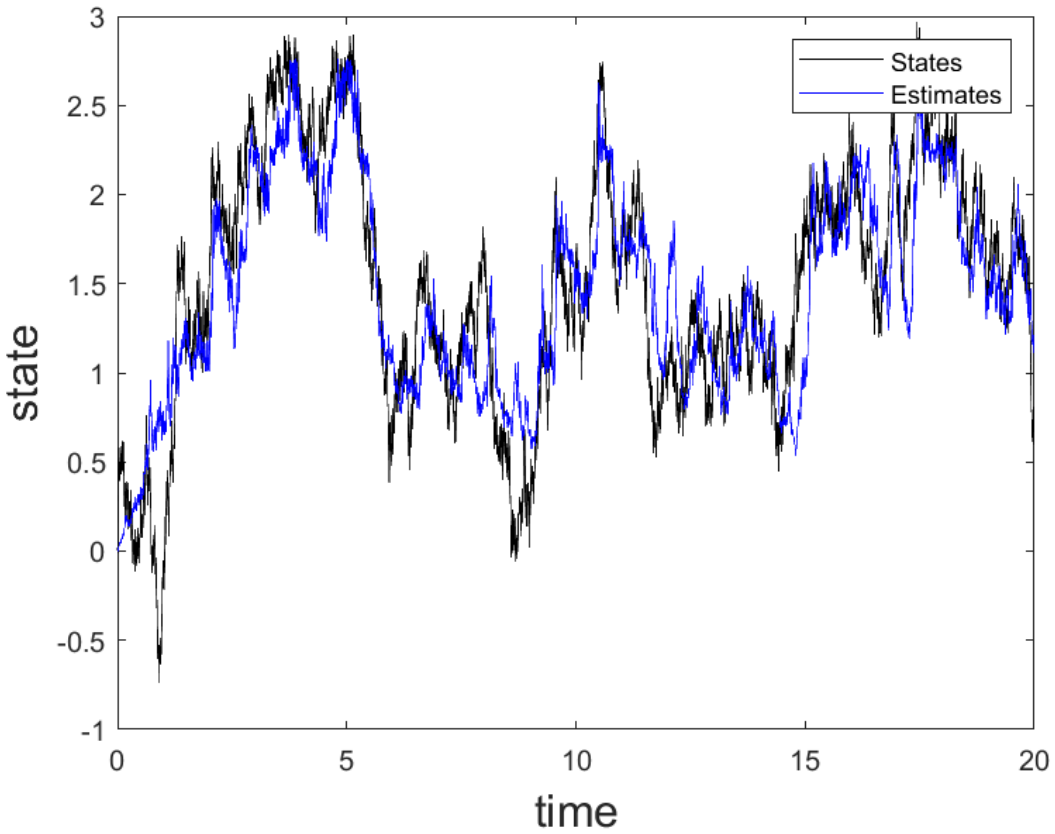
```
function U = DST_Solver(Dim, Lambda, B, U, n)
U = dstn(B*U);  % 将状态 U 转换到频域
U = U./Lambda;  % 在频域中进行元素逐次相除 (相当于逆运算)
U = dstn(U);    % 再次转换回空间域

% 处理负值与归一化
U = U(:);
U( U < 0 ) = 0;
U = U / sum(U);
```

Golub and Van Loan, Matrix Computations, *Johns Hopkins University Press*, 2012.

Result1

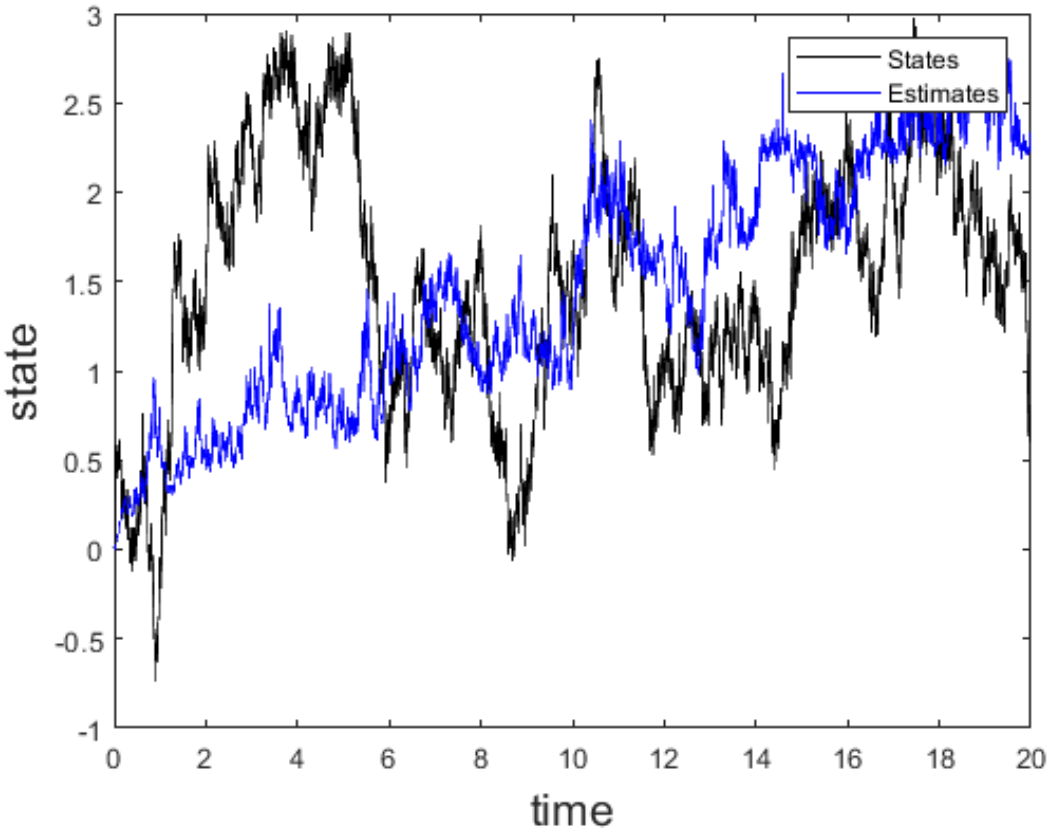
Ex. $T = 20, \quad \Delta\tau = 5\Delta t = 0.005, \quad \Delta s = 0.5, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$
20000 state, 20000 $\rightarrow$ 4000 obser, 20000 lu.



Parameter	Value
Terminal Time	20
Space Range	[-0.73856, 2.9679]
Size of Time Steps	0.001
Size of Space Steps	0.5
Root-Mean-Square Error	0.3572
Mean Error	0.27997
Time Costs	1.8074 seconds

Result2

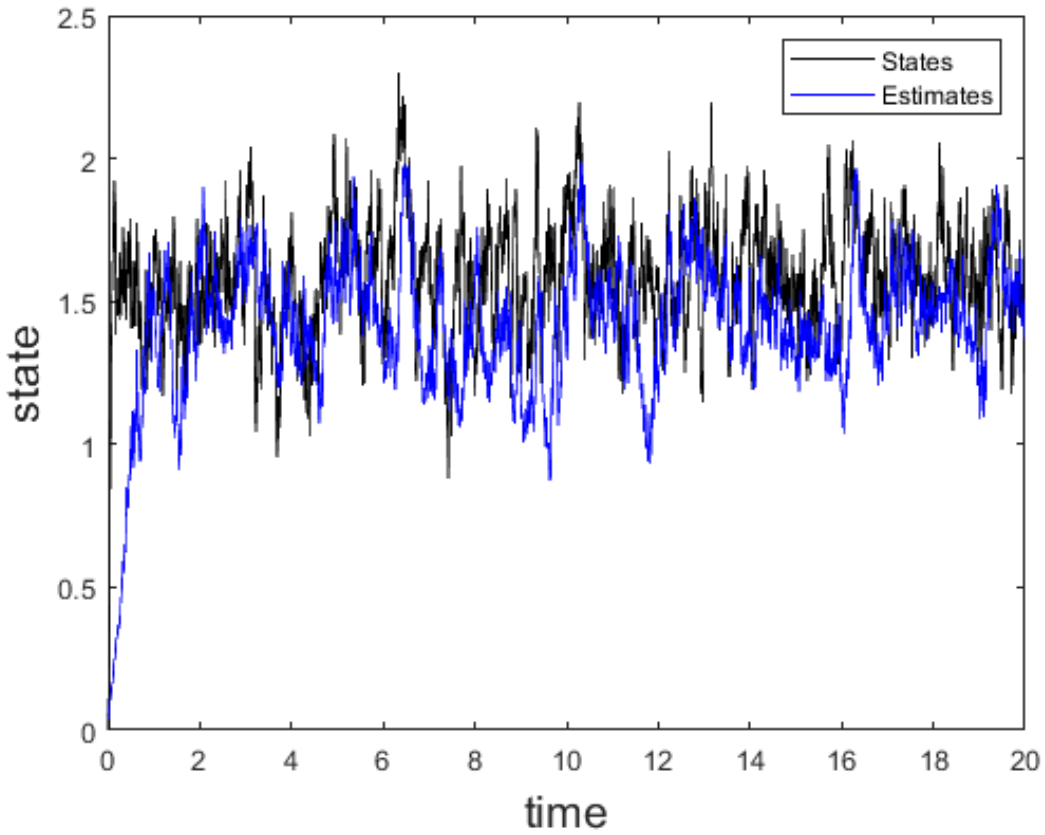
Ex. $T = 20, \quad \Delta\tau = 5\Delta t = 0.005, \quad \Delta s = 0.5, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$
20000 state, 4000 obser, 20000 lu.



Parameter	Value
Terminal Time	20
Space Range	[-0.73856, 2.9679]
Size of Time Steps	0.001
Size of Space Steps	0.5
Root-Mean-Square Error	0.87702
Mean Error	0.69526
Time Costs	1.8517 seconds

Result3

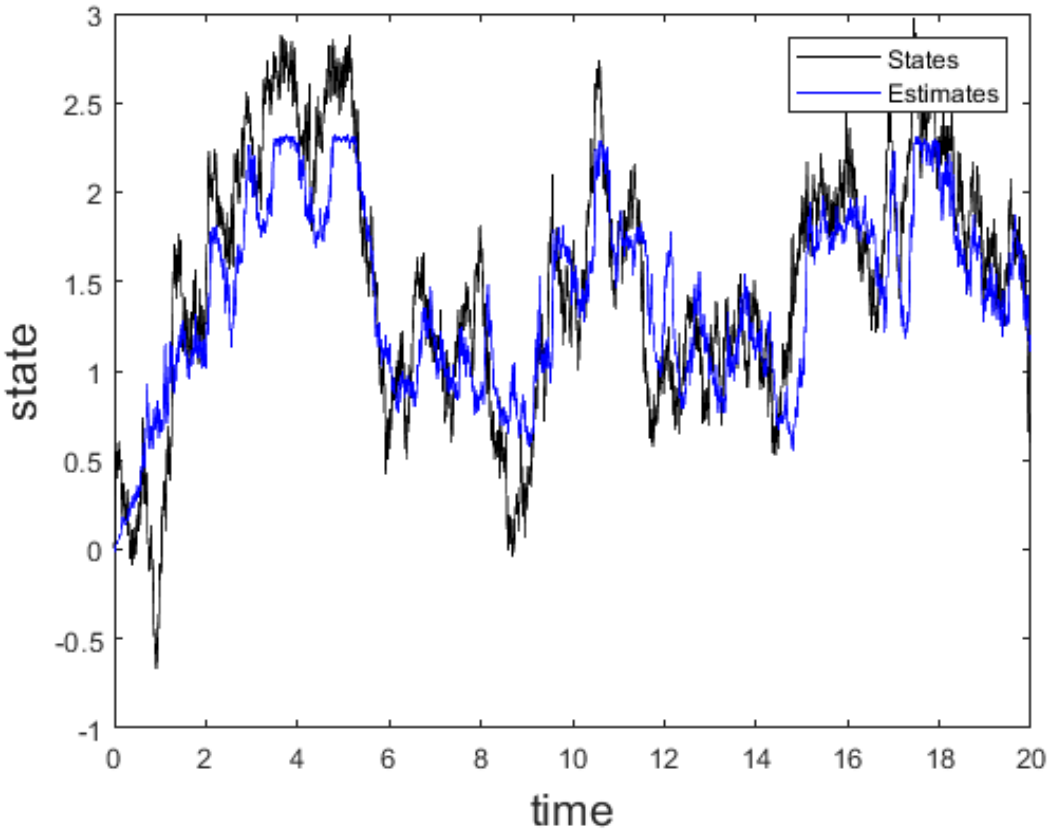
Ex. $T = 20, \quad \Delta\tau = 5 = 0.005, \quad \Delta s = 0.5, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$
4000 state, 4000 obser, 4000 lu.



Parameter	Value
Terminal Time	20
Space Range	[0, 2.2947]
Size of Time Steps	0.001
Size of Space Steps	0.5
Root-Mean-Square Error	0.32454
Mean Error	0.24145
Time Costs	0.4063 seconds

Result4

Ex. $T = 20, \quad \Delta\tau = 5\Delta t = 0.005, \quad \Delta s = 0.5, \quad \mathbf{f}(x) = \cos(x), \quad \mathbf{h}(x) = x_1^3.$
20000 $\rightarrow$ 4000 state, 20000 $\rightarrow$ 4000 obser, 4000 lu.



Parameter	Value
Terminal Time	20
Space Range	[-0.67228, 2.9679]
Size of Time Steps	0.001
Size of Space Steps	0.5
Root-Mean-Square Error	0.38068
Mean Error	0.30334
Time Costs	0.3812 seconds

YauYau Algorithm Optimization Directions

1. **High-Dimensional Efficiency**:
   
   Chen, Sun, Tao, and Yau, A Uniform Framework of Yau-Yau Algorithm Based on Deep Learning with the Capability of Overcoming the Curse of Dimensionality, IEEE Trans. Automat. Contr. 2024.
2. **Algorithm Stability**:
   - Adaptive steps and advanced filtering.
3. **Non-negativity**:
   - Ensure non-negative results in complex conditions.
4. **Complex Boundaries**:
   - Reflective and complex geometries.
5. **New Applications**:
   - Integrate with bio-data, AI, and idopNetwork.

Yau-Yau algorithm and biological data

Biological Data $\rightarrow$ **Neural Network** $\rightarrow$ $f(\cdot)$ **(Nonlinear Function)**

*Using a **Neural Network** to learn a **nonlinear** function $f(\cdot)$ from biological data, capturing the underlying dynamics and disturbances.*

$\downarrow$

$$\begin{cases} \mathrm{d}\mathbf{x}(t) = f(x(t)) + \mathrm{d}\mathbf{v}(t), \\ \mathrm{d}\mathbf{y}(t) = \mathbf{h}(\mathbf{x}(t)) + \mathrm{d}\mathbf{w}(t). \end{cases}$$

*Model the **nonlinear** dynamics and **biological disturbances** using the learned function $f(\cdot)$ within a stochastic dynamical system.*

$\downarrow$

Utilize the **Yau-Yau Filter** to estimate hidden states $\mathbf{x}(t)$ from noisy observations $\mathbf{y}(t)$, handling **biological perturbations** effectively.

References

- Chen, Sun, Tao, and Yau, A Uniform Framework of Yau-Yau Algorithm Based on Deep Learning with the Capability of Overcoming the Curse of Dimensionality, *IEEE Trans. Autom. Contr.* 2024.
- Yueh, Lin, and Yau, An Efficient Numerical Method for Solving High-Dimensional Nonlinear Filtering Problems, *Commun. Inf. Syst.* 2014.
- Yueh, Lin, and Yau, An Efficient Algorithm of Yau-Yau Method for Solving Nonlinear Filtering Problems, *Commun. Inf. Syst.* 2014.
- Luo and Yau, Hermite Spectral Method to 1-D Forward Kolmogorov Equation and Its Application to Nonlinear Filtering Problems, *IEEE Trans. Autom. Contr.* 2013.
- Luo and Yau, Complete Real Time Solution of the General Nonlinear Filtering Problem Without Memory, *IEEE Trans. Autom. Contr.* 2013.
- Yau and Yau, Real Time Solution of the Nonlinear Filtering Problem without Memory II, *SIAM J. Control Optim.* 2008.
- Yau and Yau, Real Time Solution of Nonlinear Filtering Problem without Memory I, *Math. Res. Lett.* 2000.

Thank you for your attention!